



Rapport d'Avancement et Benchmark

Architecture RAG pour un Chatbot Documentaire RCAR/CNRA

Réalisé par : Dehbi Kamal

Encadré par : M. Hilmi El Mehdi

École nationale supérieure d'Arts et Métiers de Rabat

Filière : Ingénierie numérique en Data Science IA & Big Data

Année universitaire : 2025-2026

Table des matières

Table des figures	ii
Liste des tableaux	ii
1 Introduction et Contexte du Projet	1
2 Architecture Globale (Pipeline RAG)	1
2.1 Composantes de l'Architecture	1
3 Veille Technologique et Benchmark	2
3.1 Extraction de Données (Web Crawling)	2
3.2 Recherche Hybride et Reranking	3
3.3 Modèles d'Embedding (Vectorisation)	4
3.4 Base de Données Vectorielle (Vector Database)	4
3.5 Outils pour le Pipeline RAG (Framework d'Orchestration)	5
3.6 Small Language Models (SLM) / Modèles Génératifs	6
3.7 Interface Conversationnelle (Chatbot UI)	6
4 Comparaison Synthétique de l'Architecture	8
5 Conclusion	8

Table des figures

1	Architecture globale du Chatbot RAG	1
---	---	---

Liste des tableaux

1	Benchmark des Web Crawlers	3
2	Benchmark pour la Recherche Hybride & Reranking	3
3	Benchmark des modèles d'Embedding	4
4	Benchmark des Bases de Données Vectorielles	5
5	Benchmark des Frameworks RAG	5
6	Benchmark des Small Language Models (SLM)	6
7	Benchmark des solutions d'interface conversationnelle	7
8	Synthèse et Recommandation de l'architecture technologique RAG	8

1. Introduction et Contexte du Projet

Ce projet vise à concevoir et implémenter un chatbot capable de répondre aux questions des utilisateurs à partir d'une base documentaire extraite des sites web institutionnels officiels :

- **Caisse Nationale de Retraites et d'Assurances (CNRA)** : <https://www.cnra.ma/>
- **Régime Collectif d'Allocation de Retraite (RCAR)** : <https://www.rcar.ma/>

Le système reposera sur une architecture Retrieval-Augmented Generation (RAG) permettant d'améliorer la pertinence des réponses en combinant recherche sémantique et génération par un Small Language Model (SLM) adapté à un contexte de déploiement en entreprise.

Ce document présente une veille technologique et un benchmark des solutions possibles pour les différentes briques du système (vector store, modèles d'embeddings, SLM, etc.), en justifiant les choix recommandés.

2. Architecture Globale (Pipeline RAG)

Le pipeline RAG permet de combler les lacunes d'un LLM standard en ajoutant un outil de recherche documentaire. Le flux global de ce module est découpé en plusieurs blocs indispensables.

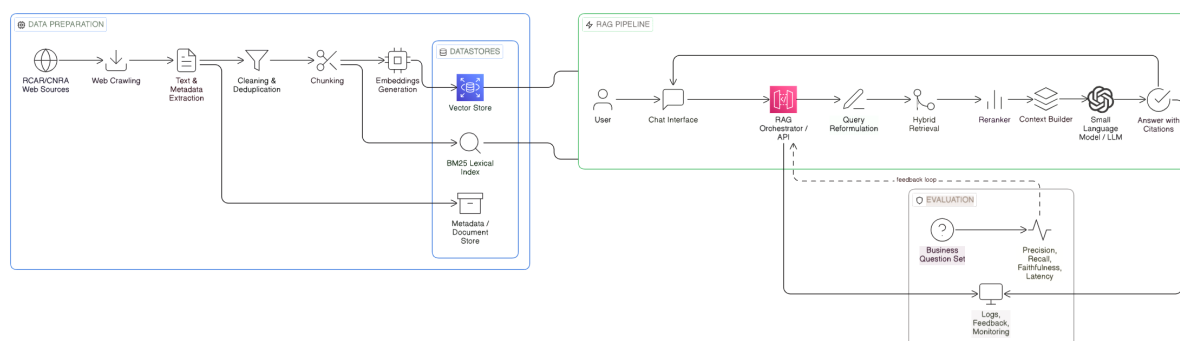


FIGURE 1 – Architecture globale du Chatbot RAG

2.1. Composantes de l'Architecture

L'architecture globale de la solution s'articule autour de trois pipelines principaux synchronisés :

1. Pipeline Offline (Préparation des Données) :

- **Web Crawling & Extraction** : Navigation automatisée sur les sources web (RCAR/CNRA) pour l'extraction du texte et des métadonnées associées.

- **Cleaning & Chunking** : Nettoyage, dédoublonnage et découpage sémantique du texte en segments de taille gérable (Chunks).
- **Embeddings & Datastores** : Transformation des segments en vecteurs (Embeddings) et stockage hybride réparti sur trois axes : Base Vectorielle (Vector Store), Index Lexical (BM25 Index) et une base de métadonnées (Document Store).

2. Pipeline Online (Chatbot RAG) :

- **Orchestration & Reformulation** : L'API intercepte la requête utilisateur via l'interface de chat et la reformule au besoin pour optimiser la recherche.
- **Recherche Hybride (Hybrid Retrieval)** : Interrogation simultanée de la base vectorielle (Sémantique) et de l'index BM25 (Mots-clés).
- **Reranker** : Un modèle spécialisé réordonne les segments obtenus pour faire remonter les plus pertinents (Top-K) au sommet.
- **Context Builder & SLM** : Construction du contexte et génération de la réponse finale par le Small Language Model (SLM), incluant les citations exactes.

3. Évaluation et Gouvernance :

- **Évaluation** : Mesure continue (Précision, Rappel, Fidélité, Latence) basée sur un ensemble de questions métier (Business Question Set).
- **Monitoring** : Boucle de feedback reliant les réponses, l'orchestrateur et les logs pour améliorer le système de manière itérative.

3. Veille Technologique et Benchmark

Cette section établit les différentes briques technologiques disponibles actuellement sur le marché et effectue un jugement comparatif pour isoler celles qui constituent le **meilleur choix (best fit)** pour un déploiement en entreprise.

3.1. Extraction de Données (Web Crawling)

La récupération et la structuration des données publiques (RCAR/CNRA) constituent la première étape du pipeline offline.

Outil	Avantages	Inconvénients
Scrapy	Standard industriel, rapide, robuste pour gros volumes.	Bas niveau, nécessite du code complexe pour le nettoyage.
BeautifulSoup / Selenium	BeautifulSoup est simple à configurer ; Selenium gère le JavaScript.	Lent à grande échelle, parcours manuel des liens.
Crawlee / Firecrawl	Frameworks modernes IA, extrait en Markdown propre, gère bien le JS.	Communauté récente, peut être lourd à héberger.

TABLE 1 – Benchmark des Web Crawlers

Le choix optimal : Crawlee ou l'utilisation d'outils modernes orientés LLM (comme Firecrawl en setup local). Ils permettent d'obtenir un texte Markdown structuré et propre avec un minimum d'effort comparé aux classiques comme BeautifulSoup, facilitant considérablement la suite du pipeline.

3.2. Recherche Hybride et Reranking

Le passage d'un RAG classique à un système robuste s'effectue via l'introduction de l'indexation hybride (BM25 + Dense) et du Reranking (réordonnancement).

Méthode / Modèle	Avantages	Inconvénients
Cohere Rerank (API)	Précision excellente, pionnier du secteur.	API commerciale, dépendance Cloud.
BM25	Ultra-rapide, idéal pour mots-clés exacts (ex : acronymes).	Ignore le sens global ou contextuel des phrases.
BAAI/bge-reranker	Leader open-source local, très bon en multilingue.	Consommation CPU/GPU supplémentaire.

TABLE 2 – Benchmark pour la Recherche Hybride & Reranking

Le choix optimal : BM25 couplé au BAAI/bge-reranker. Cette combinaison locale permet d'interroger à la fois par mots-clés exacts (BM25) et par sens (Embeddings), avant d'être

magistralement triée par le *bge-reranker* pour assurer l'extraction parfaite du contexte, tout en conservant les données *On-Premise*.

3.3. Modèles d'Embedding (Vectorisation)

Les modèles d'embedding traduisent un texte en un tableau numérique, crucial pour une recherche sémantique de qualité.

Modèle	Avantages	Inconvénients
OpenAI (text-embedding-3)	Excellente précision, usage via API simple.	Coût récurrent, données envoyées sur le Cloud.
Sentence-Transformers	Gratuit, léger, exécution 100% locale.	Précision parfois limitée en français.
BAAI/bge-m3	Leader actuel, performant en multilingue, exécution locale.	Modèle plus lourd, nécessite davantage de RAM.

TABLE 3 – Benchmark des modèles d'Embedding

Le choix optimal : BAAI/bge-m3 (HuggingFace). La confidentialité des données institutionnelles et le coût nul par inférence de ce modèle en font une option supérieure pour un projet sans concessions sur le multilinguisme.

3.4. Base de Données Vectorielle (Vector Database)

La "Vector DB" doit être rapide et capable d'évoluer avec la volumétrie des données extraites. Elle stocke les représentations sémantiques (embeddings) des documents institutionnels et assure la recherche de similarité en temps réel avec des temps de latence minimaux.

Base de Données	Avantages	Inconvénients
Pinecone / Weaviate	Puissant, managé (Cloud), très scalable.	Payant, dépendance cloud externe.
ChromaDB	Ultra-léger, idéal pour le prototypage rapide en Python.	Limité pour la très haute volumétrie en production.
Qdrant / Milvus	Open source, très performant pour un portage en production locale.	Configuration de l'infrastructure plus complexe.

TABLE 4 – Benchmark des Bases de Données Vectorielles

Le choix optimal : ChromaDB pour amorcer la première version et le pipeline RAG de prototypage, suivi d'une migration probable sur **Qdrant** au moment du déploiement en production entreprise.

3.5. Outils pour le Pipeline RAG (Framework d'Orchestration)

L'orchestrateur cimente l'application, reliant le prompt de l'utilisateur, l'indexation, la base vectorielle et le modèle de génération.

Framework	Avantages	Inconvénients
LangChain	Standard, riche en composants, conçu pour les chaînes et agents complexes.	Courbe d'apprentissage forte, format opaque.
LlamaIndex	Ultra-optimisé pour l'ingestion de données (PDFs complexes) et la recherche.	Moins généraliste pour des agents autonomes distants.

TABLE 5 – Benchmark des Frameworks RAG

Le choix optimal : LangChain combiné avec **LlamaIndex**. LlamaIndex gérera l'ingestion avancée des données web, tandis que LangChain s'impose pour introduire des logiques de routage et préparer l'intégration d'agents spécialisés (aspect Agentique) capables d'interagir avec des services externes.

3.6. Small Language Models (SLM) / Modèles Génératifs

Conformément aux exigences d'un déploiement en entreprise sécurisé, nous privilégions les Small Language Models (SLM) pouvant s'exécuter localement aux immenses LLMs commerciaux.

Modèle	Avantages	Inconvénients
GPT-4 / Claude (API)	Raisonnement exceptionnel, pas besoin de GPU puissant en local.	Problème de confidentialité, coûts récurrents (API).
Mistral 7B	Poids plume, très doué en français (natif), hébergeable localement.	Raisonnement inférieur face aux modèles 70B+.
Llama-3 (8B)	Génération véloce (via Ollama), très doué pour les instructions strictes.	Support du français perfectible selon la version.

TABLE 6 – Benchmark des Small Language Models (SLM)

Le choix optimal : Mistral 7B ou Llama-3 8B (Hébergés localement via Ollama). Ces SLMs garantissent la totale confidentialité des données exigée en entreprise. Leurs performances sont très satisfaisantes pour la restitution d'informations orientées RAG, avec des besoins matériels (VRAM) maîtrisés.

3.7. Interface Conversationnelle (Chatbot UI)

Conformément au cahier de charge, une interface conversationnelle doit permettre l'interaction en temps réel avec le pipeline RAG. Deux approches sont pertinentes : un front rapide avec Streamlit, ou une interface web complète avec React.js.

Solution	Avantages	Inconvénients
Streamlit	Développement très rapide, intégration native Python, idéal pour démonstration/MVP.	Personnalisation UI limitée, moins adapté aux usages front entreprise avancés.
React.js (Frontend) + API Python	UI moderne et totalement personnalisable, meilleure séparation Front/Back, plus robuste en production.	Implémentation plus longue, nécessite gestion complète API, auth et déploiement web.

TABLE 7 – Benchmark des solutions d’interface conversationnelle

Le choix optimal : Usage de **Streamlit** pour livrer rapidement une première version fonctionnelle du chatbot, puis migrer vers une interface **React.js** pour une industrialisation (UX avancée, authentification, monitoring front, intégration SI).

4. Comparaison Synthétique de l'Architecture

Le tableau synthétique ci-dessous résume de façon abrégée les solutions explorées et encadre le choix définitif de la première architecture technique recommandée pour le projet.

Composant du Pipeline	Options (Shortlist)	Choix d'Implémentation (Best Fit)
Web Crawling	Scrapy, BeautifulSoup, Crawlee	Crawlee : Extrait un format Mark-down propre adapté aux LLMs.
Modèles d'Embeddings	OpenAI SDK, S-Transformers, BAAI	BAAI/bge-m3 : Local, précis en français, aucun coût d'API.
Base de Données Vectorielle	ChromaDB, Qdrant, Pinecone	ChromaDB pour prototypage, Qdrant pour la production.
Recherche Hybride	Cohere, Elasticsearch, BAAI	BM25 + BAAI/bge-reranker : Allie mots-clés exacts et sémantique.
Orchestration	LangChain, LlamaIndex, Haystack	LangChain + LlamaIndex : Ingestion de données puis agents.
Générateur (SLM)	GPT-4 API, Mistral 7B, Llama-3 8B	Mistral 7B ou Llama 3 8B (Ollama) : Assure la confidentialité.

TABLE 8 – Synthèse et Recommandation de l'architecture technologique RAG

5. Conclusion

Considérant les impératifs de souveraineté des données, de fiabilité et d'évolutivité d'un déploiement en entreprise au service public (RCAR/CNRA), il en résulte que la pile **100% Open Source Locale (BAAI + ChromaDB + LangChain + Mistral/Llama-3)** assure la totalité de la chaîne fonctionnelle demandée. Elle répond efficacement au besoin de recherche sémantique via le pipeline RAG et pave la voie, grâce à LangChain, au développement futur d'agents spécialisés connectés aux services externes.